

DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

Ajuste fino de hiperparámetros

TLY1102

FORMAMOS **PERONAS** PARA UNA **SOCIEDAD** MEJOR

• ¿Qué hemos logrado hasta ahora?

Durante las sesiones anteriores desarrollamos un **Pipeline Integrado**, capaz de:

- Cargar y analizar un conjunto de datos.
- Realizar el preprocesamiento y aumento de datos.
- Construir modelos MLP y CNN.
- Incorporar técnicas de regularización.
- Evaluar el desempeño mediante múltiples métricas.
- Auditar el comportamiento del modelo utilizando herramientas de Explainable AI.

Al finalizar la Unidad 3 disponemos de un modelo funcional y correctamente evaluado.



• ¿Ha terminado el proyecto?

No necesariamente.

Un modelo funcional no implica que sea el mejor modelo posible.

Siempre es posible mejorar aspectos como:

- El rendimiento predictivo.
- La capacidad de generalización.
- La estabilidad del entrenamiento.
- La robustez frente a nuevos datos.
- La eficiencia computacional.

Estas mejoras forman parte de la **etapa de optimización** del modelo.



Entrenar un modelo es solo el comienzo. Optimizarlo consiste en realizar mejoras sistemáticas y fundamentadas para obtener una solución más robusta y confiable.



01

Optimización

¿Qué significa optimizar un modelo?

Optimizar es diferente a Aumentar Accuracy solamente.

¿Qué SÍ significa optimizar?

- Formular una hipótesis.
- Modificar una variable a la vez.
- Medir el impacto del cambio.
- Comparar los resultados con evidencia experimental.
- Conservar únicamente aquellas modificaciones que realmente mejoren el modelo.

¿Qué NO significa optimizar?

- Probar hiperparámetros al azar.
- Cambiar diez cosas al mismo tiempo.
- Buscar únicamente la mayor Accuracy.

¿Qué podemos optimizar?

Hiperparámetros

Durante el entrenamiento de una red neuronal existen dos tipos de valores que determinan el comportamiento del modelo:

- **Parámetros:** son aprendidos automáticamente durante el entrenamiento.
- **Hiperparámetros:** son definidos por el desarrollador antes de comenzar el entrenamiento.

Los hiperparámetros controlan cómo aprenderá el modelo, mientras que los **parámetros corresponden al conocimiento que el modelo adquiere** a partir de los datos.

Parámetros	Hiperparámetros
Se aprenden automáticamente.	Son definidos por el usuario.
Cambian durante el entrenamiento.	Permanecen fijos durante cada entrenamiento.
Se optimizan mediante descenso del gradiente.	Se ajustan mediante experimentación.
Ejemplo: pesos (weights) y sesgos (bias).	**Ejemplo: Learning Rate, Batch Size, número de capas**.



02

Hiperparámetros

• Principales hiperparámetros

Durante el diseño de un modelo existen numerosos hiperparámetros que pueden modificarse para mejorar su desempeño. Los más utilizados se muestran en la siguiente tabla.

Hiperparámetro	¿Qué controla?	Impacto sobre el entrenamiento
Learning Rate	Tamaño de cada actualización del gradiente.	Controla la velocidad de aprendizaje y la estabilidad de la convergencia.
Batch Size	Número de muestras utilizadas antes de actualizar los pesos.	Afecta el tiempo de entrenamiento, el consumo de memoria y la estabilidad del gradiente.
Número de Epochs	Cantidad de veces que el modelo recorre completamente el conjunto de entrenamiento.	Influye en el aprendizaje y el riesgo de overfitting.
Número de capas	Profundidad de la red neuronal.	Determina la capacidad del modelo para aprender representaciones complejas.
Número de neuronas / filtros	Complejidad de cada capa.	Modelos más grandes pueden aprender relaciones más complejas, pero también sobreajustarse.
Dropout	Porcentaje de neuronas desactivadas durante el entrenamiento.	Reduce el overfitting y mejora la generalización.
Regularización L2	Penalización aplicada a los pesos del modelo.	Favorece modelos más simples y robustos.
Optimizer	Algoritmo utilizado para actualizar los pesos.	Influye en la velocidad y calidad de la convergencia.

• Clasificación de los hiperparámetros

No todos los hiperparámetros tienen el mismo impacto. Algunos modifican la **arquitectura** del modelo, mientras que otros afectan únicamente el **proceso de entrenamiento**.

Podemos agrupar los hiperparámetros según la etapa del Pipeline que modifican.

Categoría	Ejemplos	¿Qué modifican?
Arquitectura	Número de capas, número de neuronas, filtros, funciones de activación.	La estructura del modelo.
Entrenamiento	Learning Rate, Batch Size, Epochs, Optimizer.	El proceso de aprendizaje.
Regularización	Dropout, L2, Batch Normalization.	La capacidad de generalización del modelo.
Preprocesamiento	Normalización, tamaño de imagen, Data Augmentation.	La calidad de los datos de entrada.

Optimizar un modelo no consiste únicamente en modificar el Learning Rate. También implica decidir qué arquitectura utilizar, cómo entrenarla y qué técnicas aplicar para mejorar su capacidad de generalización.

¿Cómo encontramos buenos hiperparámetros?

La optimización consiste en comparar distintas configuraciones del modelo utilizando un procedimiento objetivo y reproducible.

Una vez definidos los hiperparámetros del modelo, surge una nueva pregunta:

¿Qué valores producen el mejor desempeño?

Por ejemplo, supongamos que deseamos optimizar los siguientes hiperparámetros:

Hiperparámetro	Posibles valores
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64
Dropout	0.2, 0.4, 0.5
Número de capas	2, 3, 4

¿Cuál combinación deberíamos utilizar?

No existe una respuesta universal

La mejor configuración depende de:

- El conjunto de datos.
- La arquitectura utilizada.
- El problema a resolver.
- La función de pérdida.
- El optimizador.

Por esta razón, los hiperparámetros deben determinarse mediante **experimentación controlada**.

• Estrategias para buscar hiperparámetros

Existen distintos métodos para explorar el espacio de búsqueda

Método	¿Cómo funciona?	Ventajas	Desventajas
Grid Search	Evalúa todas las combinaciones posibles.	Encuentra la mejor combinación dentro de la grilla definida.	Alto costo computacional cuando existen muchos hiperparámetros.
Random Search	Evalúa combinaciones seleccionadas aleatoriamente.	Explora un espacio más amplio con menos experimentos.	No garantiza evaluar todas las combinaciones posibles.
Bayesian Optimization	Utiliza los resultados previos para decidir qué combinación probar después.	Muy eficiente para problemas complejos.	Mayor complejidad de implementación.

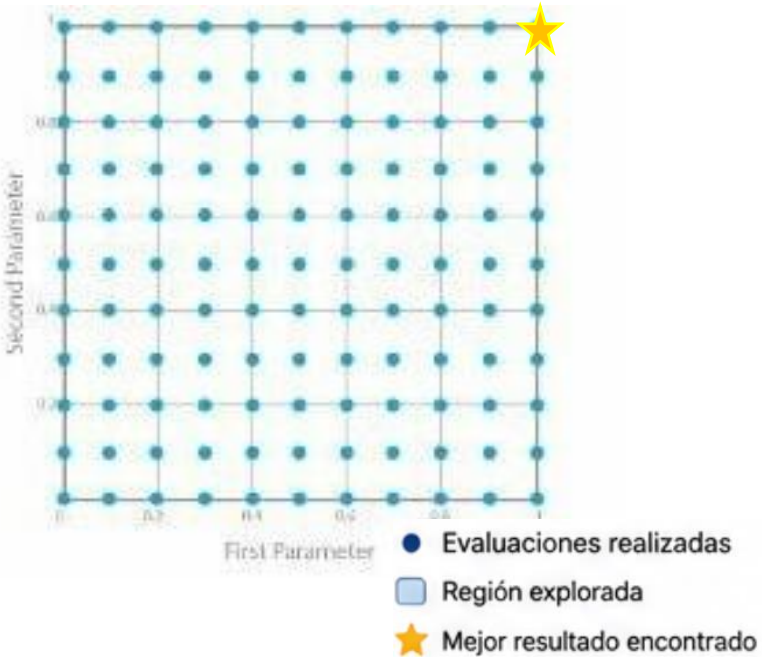
No existe un método mejor para todos los problemas.
La elección dependerá del tiempo disponible, el costo computacional y el tamaño del espacio de búsqueda.

Grid Search

Grid Search prueba **todas las combinaciones posibles** de los hiperparámetros definidos por el desarrollador.

```
from sklearn.model_selection import GridSearchCV

GridSearchCV(
    estimator=modelo,
    param_grid=parametros,
    cv=5
)
```



Ventajas

- Fácil de implementar.
- Reproducible.
- Encuentra la mejor combinación dentro del conjunto evaluado.

Desventajas

- El número de modelos crece rápidamente.
- Requiere mucho tiempo para redes profundas.

Random Search

En lugar de evaluar todas las combinaciones posibles, Random Search selecciona un subconjunto de configuraciones de forma aleatoria.

```
from sklearn.model_selection
import RandomizedSearchCV

RandomizedSearchCV(
    estimator=modelo,

param_distributions=parametros,
    n_iter=20,
    cv=5
)
```



Ventajas

- Mucho más rápido.
- Permite explorar regiones más amplias del espacio de búsqueda.
- En muchos problemas obtiene resultados similares a Grid Search utilizando menos experimentos.

Desventajas

- No garantiza encontrar la mejor combinación posible.
- Los resultados pueden variar entre ejecuciones.

Bayesian Search

Es un método de **búsqueda inteligente** que **utiliza los resultados obtenidos en experimentos anteriores** para decidir qué hiperparámetros probar después.

A diferencia de Grid y Random Search, **no prueba combinar al azar ni todas las combinaciones posibles**.

Su objetivo es encontrar la mejor combinación de hiperparámetros utilizando el menor número posible de entrenamientos del modelo.

¿Cómo funciona?



1 Modelo probabilístico

Se construye un modelo (por ejemplo, Gaussian Process) que estima la relación entre los hiperparámetros y el desempeño (obtenido hasta ahora).

2 Función de adquisición

Se utiliza una función que balancea exploración (probar nuevas regiones) y explotación (probar donde ya se obtuvo buenos resultados).

3 Selección del siguiente punto

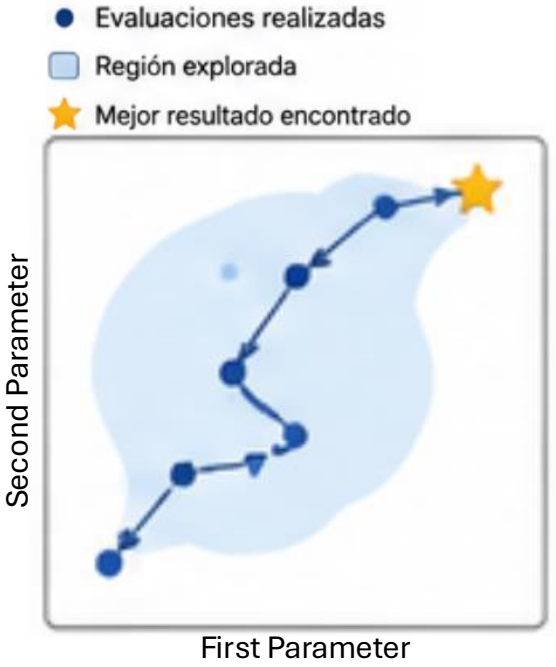
Se elige la combinación de hiperparámetros que maximiza la función de adquisición.

4 Entrenamiento y evaluación

Se entrena el modelo con esa combinación, se evalúa (usando validación cruzada) y el resultado se incorpora al modelo.

5 Iteración

El proceso se repite hasta cumplir un criterio de parada (tiempo, número de iteraciones, convergencia, etc.).



Duoc UC®

Bayesian Search

Ventajas

- Muy eficiente en espacios de búsqueda grandes o costosos.
- Requiere mucho menos entrenamiento que Grid Search.
- Encuentra buenas soluciones rápidamente.
- Se adapta dinámicamente a la información obtenida.

Desventajas

- Más compleja de implementar.
- Puede ser menos efectiva en espacios de búsqueda muy ruidosos.
- Requiere elección de componentes (modelo probabilístico y función de adquisición).

```
import keras_tuner as kt

# Definir el modelo y los hiperparámetros
def build_model(hp):
    ...

# Crear el optimizador bayesiano
tuner = kt.BayesianOptimization(
    build_model,
    objective="val_accuracy",
    max_trials=20
)

# Buscar la mejor configuración
tuner.search(x_train, y_train)

# Obtener los mejores hiperparámetros
best_hp = tuner.get_best_hyperparameters()[0]
```



03

Escoger la mejor configuración

• ¿Cómo sabemos cuál configuración es realmente mejor?

Supongamos que Grid Search encontró la siguiente configuración:

- Learning Rate = 0.001
- Batch Size = 32
- Dropout = 0.3

Y obtuvo:

```
Accuracy = 91.4%
```

¿Podemos asegurar que esa configuración es realmente la mejor?
No necesariamente.

El resultado puede depender del conjunto de entrenamiento y validación utilizado.

Necesitamos una forma más robusta de comparar los distintos experimentos.

La Validación Cruzada permite estimar el desempeño del modelo utilizando distintas particiones del conjunto de datos, reduciendo la dependencia de una única división Train/Test.

Validación Cruzada (Cross Validation)

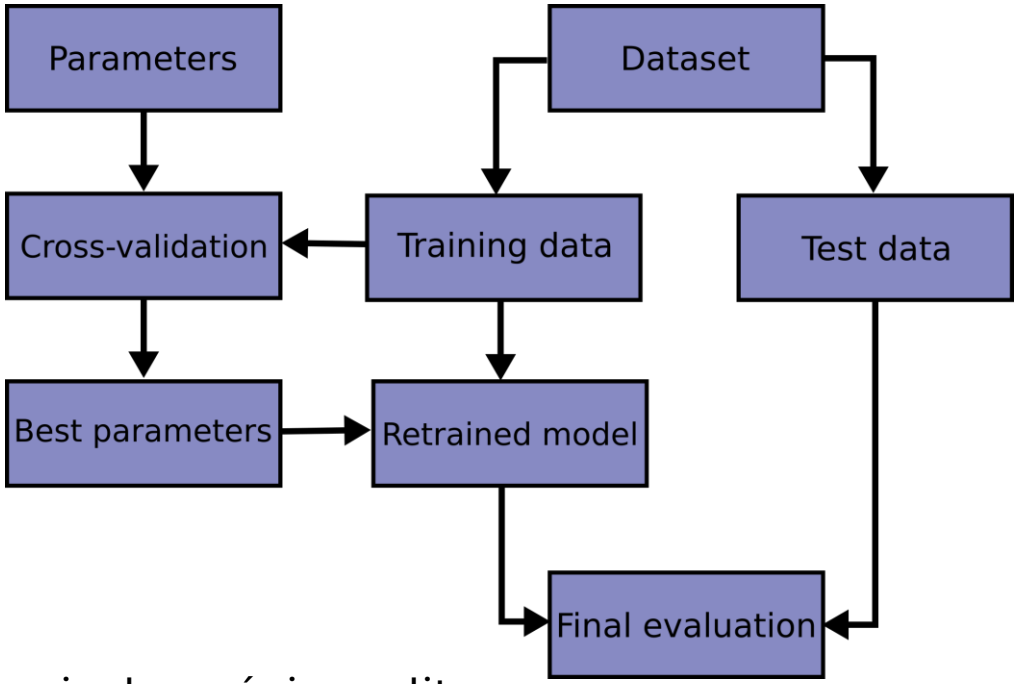
En lugar de evaluar el modelo utilizando una única división entre entrenamiento y prueba, el conjunto de datos se divide varias veces.

El modelo se entrena y evalúa repetidamente, obteniendo una estimación más estable de su desempeño.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=0)

X_train.shape, y_train.shape
X_test.shape, y_test.shape

clf = svm.SVC(kernel='linear', C=1).fit(X_train, y_train)
clf.score(X_test, y_test)
```



Beneficios

- Reduce la dependencia de un único split.
- Produce una estimación más robusta.
- Permite comparar modelos de forma más confiable.

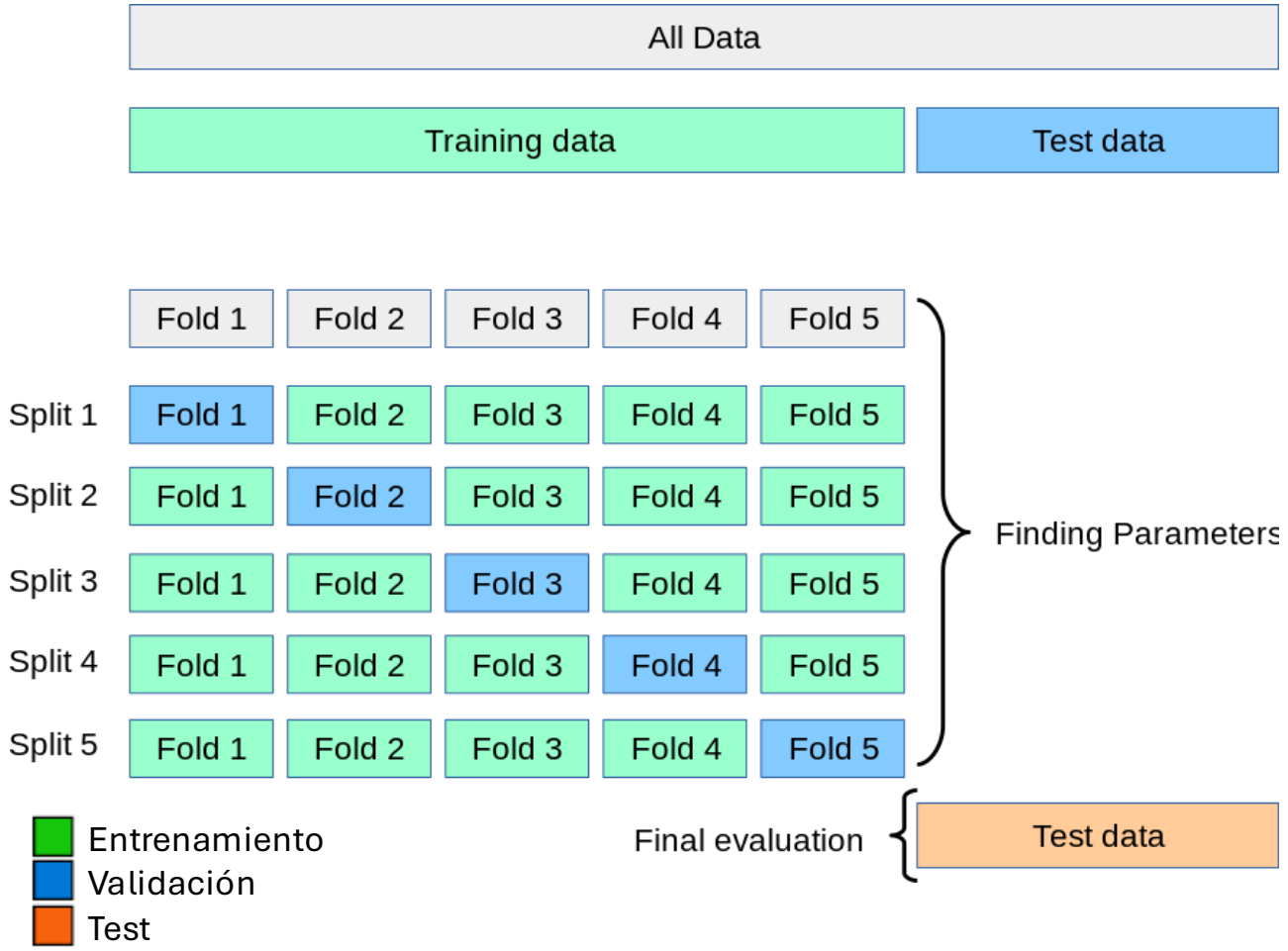
K-Fold Cross Validation

El conjunto de datos se divide en **K particiones (folds)** del mismo tamaño.

En cada iteración:

- Un fold se utiliza para validación.
- Los restantes se utilizan para entrenamiento.

El proceso se repite hasta que todos los folds hayan sido utilizados como conjunto de validación.

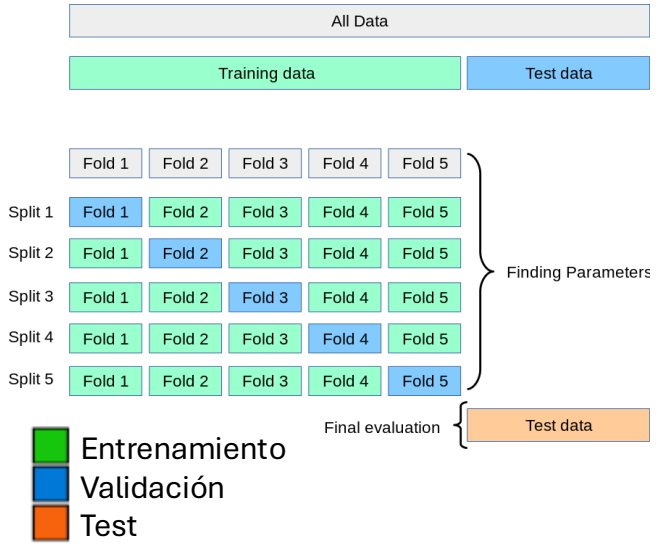


ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

K-Fold Cross Validation

Funcionamiento

1. Se separa el conjunto de prueba (**Test Set**), el cual permanecerá sin utilizar durante todo el proceso de optimización.
2. El conjunto de entrenamiento se divide en **K particiones (folds)** del mismo tamaño.
3. En cada iteración:
   - un fold se utiliza como conjunto de **validación**;
   - los restantes se utilizan para **entrenamiento**.
4. El proceso se repite hasta que cada fold haya sido utilizado una vez como conjunto de validación.
5. Finalmente se calcula el **promedio de las métricas** obtenidas en las K iteraciones, proporcionando una estimación más robusta del desempeño del modelo.



DuocUC®

¿Se utiliza siempre K-Fold en Deep Learning?

En modelos clásicos

Muy **recomendable**.

- Regresión.
- SVM.
- Árboles.
- Random Forest.

En Deep Learning

Generalmente **no**.

¿Por qué?

- El entrenamiento puede tardar horas o días.
- K-Fold implica entrenar el modelo K veces.
- El costo computacional aumenta considerablemente.

Por esta razón, en Deep Learning suele utilizarse:

- Train / Validation / Test.
- Validation Split.
- Early Stopping.

La Validación Cruzada se reserva para:

- Conjuntos de datos pequeños.
- Comparación de arquitecturas.
- Evaluación experimental.



04

Detener la optimización del modelo

- # ¿Existe un punto donde el modelo es "suficientemente bueno"?

Durante el proceso de optimización es común pensar que **siempre es posible obtener una Accuracy ligeramente mayor**. Sin embargo, **en un proyecto real llega un momento** en que **continuar optimizando deja de ser conveniente**.

Cada nueva mejora requiere más tiempo, **mayor costo computacional** y **aumenta la complejidad del sistema**, **mientras que el beneficio obtenido puede ser muy pequeño**.

Por esta razón, **la optimización debe detenerse cuando el costo de seguir mejorando supera el beneficio esperado**.

La optimización implica encontrar un equilibrio (Trade-off)

Seguir optimizando...	Posibles consecuencias
Más épocas de entrenamiento	Mayor tiempo de ejecución y riesgo de overfitting.
Arquitecturas más profundas	Incremento del costo computacional y del número de parámetros.
Más hiperparámetros por ajustar	Mayor cantidad de experimentos y tiempo de desarrollo.
Modelos más complejos	Menor interpretabilidad y mayor dificultad de mantenimiento.

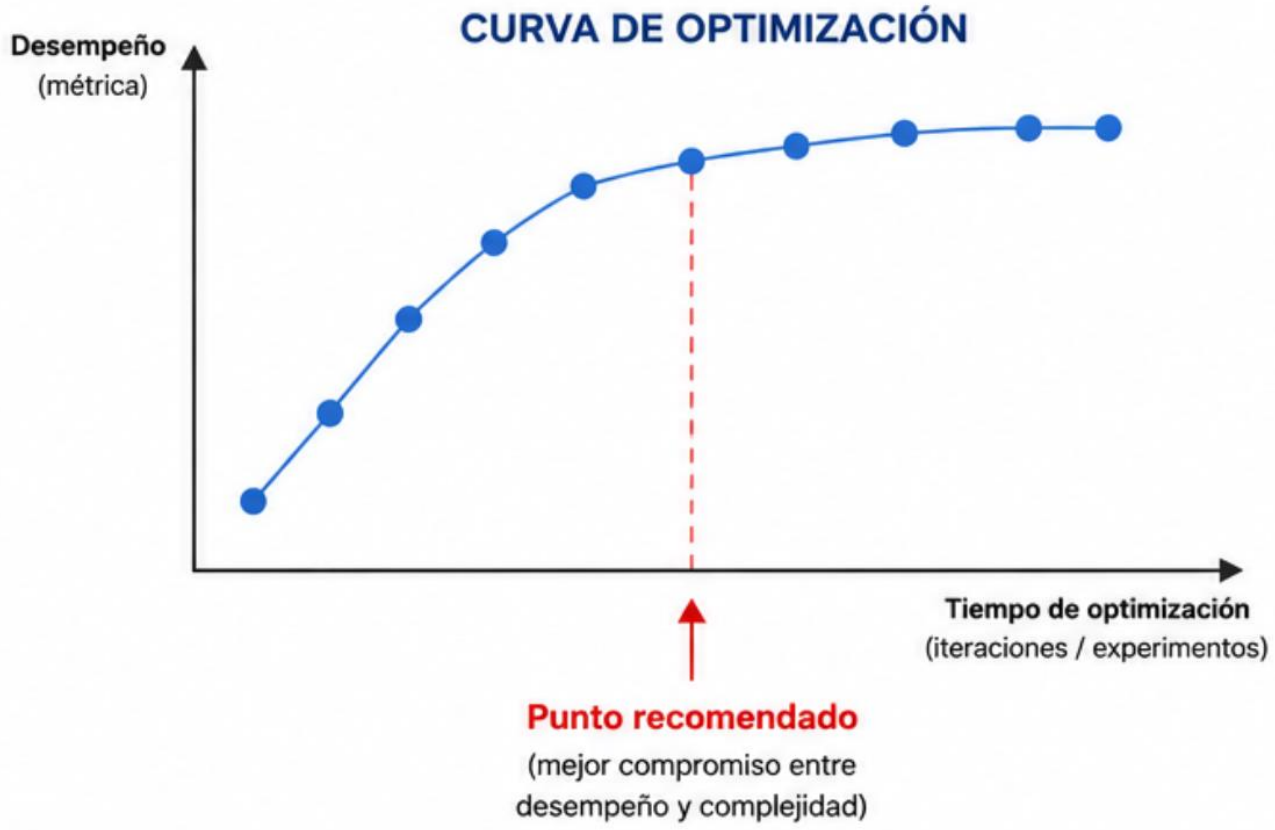
¿Existe un punto donde el modelo es "suficientemente bueno"?

¿Cómo decidir cuándo detener la optimización?

Antes de realizar una nueva iteración pregúntate:

- ¿La mejora observada es realmente significativa?
- ¿El modelo sigue generalizando correctamente?
- ¿Las métricas son consistentes en validación y prueba?
- ¿La auditoría muestra un comportamiento confiable?
- ¿El costo computacional adicional se justifica?

Si la respuesta es **no**, probablemente el modelo ya alcanzó un nivel de desempeño adecuado para el problema.



¿Existe un punto donde el modelo es "suficientemente bueno"?

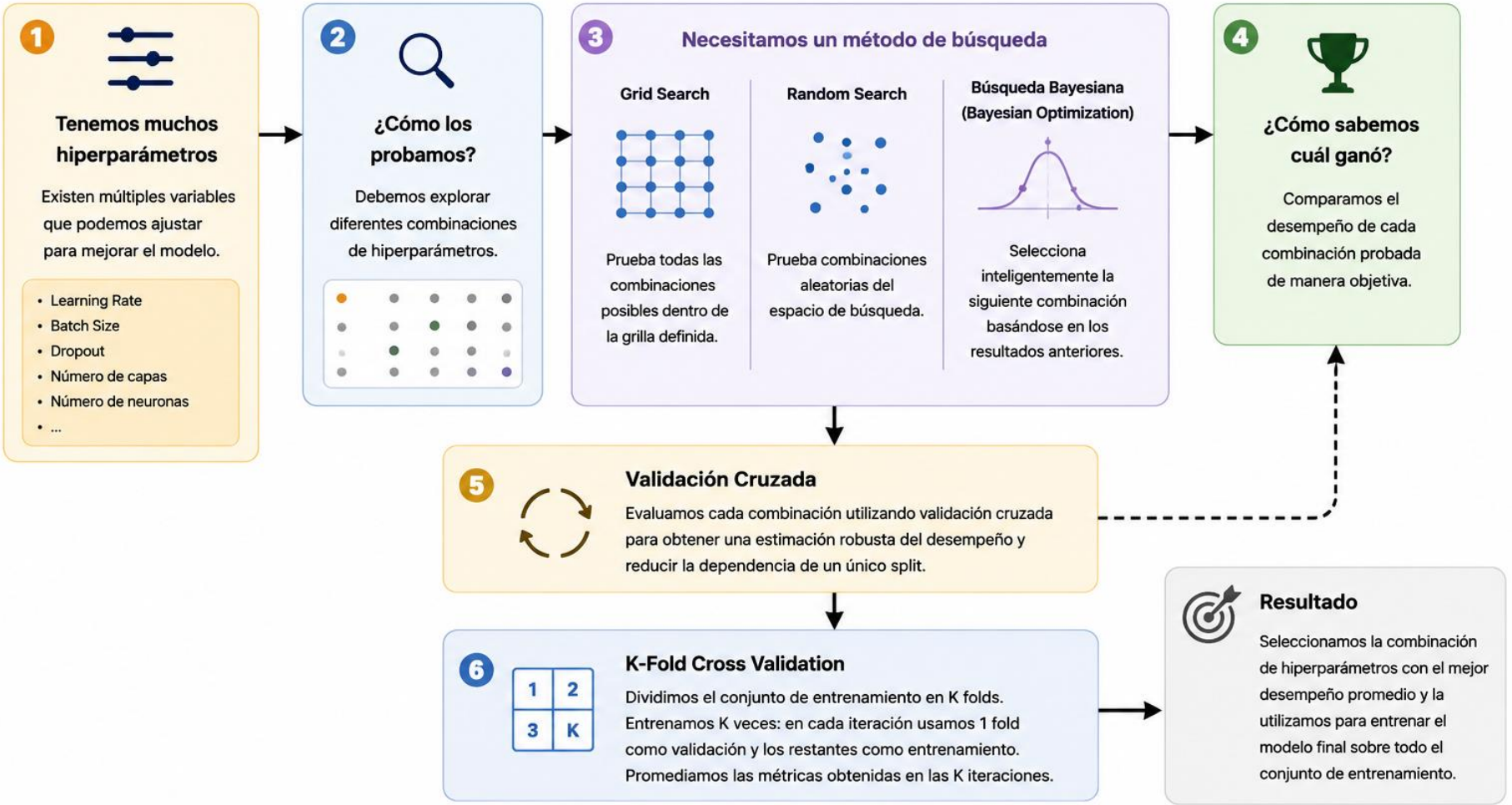
Un buen ingeniero no busca el modelo más complejo; busca el modelo más adecuado para resolver el problema.

El objetivo de la optimización no es obtener el modelo más complejo, sino el modelo que entregue el mejor equilibrio entre desempeño, capacidad de generalización, costo computacional e interpretabilidad.

Buenas prácticas antes de dar por finalizado un modelo

- Las curvas de entrenamiento y validación muestran una buena capacidad de generalización.
- No existe evidencia importante de overfitting.
- Las métricas (Accuracy, Precision, Recall, F1, ROC-AUC) son satisfactorias.
- La auditoría del modelo (matriz de confusión, XAI, análisis de errores) respalda las decisiones del modelo.
- El tiempo de entrenamiento y el costo computacional son razonables para el problema.
- Las mejoras obtenidas en las últimas iteraciones son pequeñas o marginales.

• Esqueme de optimización de hiperparámetros



El objetivo es encontrar la combinacion de hiperparámetros que maximice el desempeño del modelo con buena capacidad de generalización y un costo computacionalrazonable.

Duoc UC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.